

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL

MARKS: 50

Annexure A

Coding Challenge

Code of Conduct:

I Challa Priyadarshini(192311118)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Ch. Priyadarshini

Annexure A

Coding Challenge

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.
2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.
3. Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.
4. Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.
5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

Question 1:

```
unsigned int generateMAC(char *message)
{
    unsigned int mac = 0;
    int i;
    for (i = 0; message[i] != '\0'; i++)
    {
        mac = mac ^ ((unsigned char)message[i] * KEY);
        mac = mac + (unsigned char)message[i];
    }
    return mac;
}

int main()
{
    char packet[200];
    char receivedPacket[200];
    unsigned int generatedMAC;
    unsigned int receivedMAC;
    printf("==== IPsec Authentication Header (AH) Simulation ==== \n");
    printf("\nEnter IP Packet Data: ");
    fgets(packet, sizeof(packet), stdin);
    packet[strcspn(packet, "\n")] = '\0';
    generatedMAC = generateMAC(packet);
    printf("\n--- Sender --- \n");
    printf("IP Packet : %s \n", packet);
    printf("Generated MAC : %u \n", generatedMAC);
    printf("\n--- Receiver --- \n");
    printf("Enter Received IP Packet: ");
    fgets(receivedPacket, sizeof(receivedPacket), stdin);
    receivedPacket[strcspn(receivedPacket, "\n")] = '\0';
    printf("Enter Received MAC: ");
    scanf("%u", &receivedMAC);
    unsigned int calculatedMAC = generateMAC(receivedPacket);
    printf("\nCalculated MAC : %u \n", calculatedMAC);
}
```

Compile Log Debug Find Results Close

Output:

```
==== IPsec Authentication Header (AH) Simulation ====

Enter IP Packet Data: Hello IPsec

--- Sender ---
IP Packet : Hello IPsec
Generated MAC : 4140

--- Receiver ---
Enter Received IP Packet: Hello IPsec
Enter Received MAC: 123456

Calculated MAC : 4140

Authentication FAILED!
Packet has been modified.

-----
Process exited after 50.53 seconds with return value 0
Press any key to continue . . . |
```

Question 2:

```
#include <stdio.h>
#include <string.h>
void encrypt(char *data, char *key)
{
    int i;
    int keyLength = strlen(key);

    for (i = 0; data[i] != '\0'; i++)
    {
        data[i] = data[i] ^ key[i % keyLength];
    }
}

void decrypt(char *data, char *key)
{
    encrypt(data, key);
}

void printHex(char *data, int length)
{
    int i;
    for (i = 0; i < length; i++)
        printf("%02X ", (unsigned char)data[i]);
    printf("\n");
}

int main()
{
    char payload[200];
    char key[100];
    char encrypted[200];
    int length;

    printf("==== IPsec ESP Simulation ====\n");
    printf("\nEnter IP Packet Payload: ");
    fgets(payload, sizeof(payload), stdin);
    payload[strcspn(payload, "\n")] = '\0';
    printf("Enter Secret Key: ");
    fgets(key, sizeof(key), stdin);
    key[strcspn(key, "\n")] = '\0';
    length = strlen(payload);
    strcpy(encrypted, payload);
    encrypt(encrypted, key);
    printf("\n--- ESP Encryption ---\n");
    printf("Original Payload : %s\n", payload);
    printf("Encrypted Payload : ");
    printHex(encrypted, length);
    decrypt(encrypted, key);
    printf("\n--- ESP Decryption ---\n");
    printf("Decrypted Payload : %s\n", encrypted);
    if (strcmp(payload, encrypted) == 0)
        printf("\nESP SUCCESS: Original payload restored.\n");
    else
        printf("\nESP FAILED: Decryption error.\n");
    return 0;
}
```

```
printf("%02X ", (unsigned char)data[i]);
printf("\n");
}

int main()
{
    char payload[200];
    char key[100];
    char encrypted[200];
    int length;

    printf("==== IPsec ESP Simulation ====\n");
    printf("\nEnter IP Packet Payload: ");
    fgets(payload, sizeof(payload), stdin);
    payload[strcspn(payload, "\n")] = '\0';
    printf("Enter Secret Key: ");
    fgets(key, sizeof(key), stdin);
    key[strcspn(key, "\n")] = '\0';
    length = strlen(payload);
    strcpy(encrypted, payload);
    encrypt(encrypted, key);
    printf("\n--- ESP Encryption ---\n");
    printf("Original Payload : %s\n", payload);
    printf("Encrypted Payload : ");
    printHex(encrypted, length);
    decrypt(encrypted, key);
    printf("\n--- ESP Decryption ---\n");
    printf("Decrypted Payload : %s\n", encrypted);
    if (strcmp(payload, encrypted) == 0)
        printf("\nESP SUCCESS: Original payload restored.\n");
    else
        printf("\nESP FAILED: Decryption error.\n");
    return 0;
}
```

Output:

```
==== IPsec ESP Simulation ====

Enter IP Packet Payload: Hello IPsec
Enter Secret Key: secret

--- ESP Encryption ---
Original Payload : Hello IPsec
Encrypted Payload : 68 16 09 0F 1D 45 3D 70 20 00 00

--- ESP Decryption ---
Decrypted Payload : Hello IPS

ESP FAILED: Decryption error.

-----
Process exited after 42.67 seconds with return value 0
Press any key to continue . . . |
```

Question 3:

```
#include <stdio.h>
#include <string.h>
#define MAX_SA 10
struct SecurityAssociation
{
    int spi;
    char srcIP[20];
    char destIP[20];
    char protocol[10];
    char encryption[20];
};
struct SecurityAssociation sa[MAX_SA];
int count = 0;
void createSA()
{
    if (count >= MAX_SA)
    {
        printf("\nSA table is full!\n");
        return;
    }
    printf("\nEnter SPI: ");
    scanf("%d", &sa[count].spi);
    printf("Enter Source IP: ");
    scanf("%s", sa[count].srcIP);
    printf("Enter Destination IP: ");
    scanf("%s", sa[count].destIP);
    printf("Enter Protocol (AH/ESP): ");
    scanf("%s", sa[count].protocol);
    printf("Enter Encryption: ");
    scanf("%s", sa[count].encryption);
    count++;
    printf("\nSecurity Association created successfully!\n");
}

void lookupSA()
{
    int spi;
    int i;
    printf("\nEnter SPI to lookup: ");
    scanf("%d", &spi);
    for (i = 0; i < count; i++)
    {
        if (sa[i].spi == spi)
        {
            printf("\n--- Security Association Found ---\n");
            printf("SPI           : %d\n", sa[i].spi);
            printf("Source IP      : %s\n", sa[i].srcIP);
            printf("Destination IP : %s\n", sa[i].destIP);
            printf("Protocol       : %s\n", sa[i].protocol);
            printf("Encryption     : %s\n", sa[i].encryption);
            return;
        }
    }
    printf("\nSecurity Association not found!\n");
}

void deleteSA()
{
    int spi;
    int i, j;
    printf("\nEnter SPI to delete: ");
    scanf("%d", &spi);
    for (i = 0; i < count; i++)
    {
        if (sa[i].spi == spi)
        {
            for (j = i; j < count - 1; j++)
            {
                sa[j] = sa[j+1];
            }
            count--;
            return;
        }
    }
    printf("\nSecurity Association not found!\n");
}

for (i = 0; i < count; i++)
{
    printf("\nSA %d\n", i + 1);
    printf("SPI           : %d\n", sa[i].spi);
    printf("Source IP      : %s\n", sa[i].srcIP);
    printf("Destination IP : %s\n", sa[i].destIP);
    printf("Protocol       : %s\n", sa[i].protocol);
    printf("Encryption     : %s\n", sa[i].encryption);
}

int main()
{
    int choice;
    while (1)
    {
        printf("\n\n===== IPsec Security Association Manager =====\n");
        printf("1. Create SA\n");
        printf("2. Lookup SA\n");
        printf("3. Delete SA\n");
        printf("4. Display All SAs\n");
        printf("5. Exit\n");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                createSA();
                break;
            case 2:
                lookupSA();
                break;
            case 3:
                deleteSA();
                break;
            case 4:
                for (i = 0; i < count; i++)
                {
                    printf("\nSA %d\n", i + 1);
                    printf("SPI           : %d\n", sa[i].spi);
                    printf("Source IP      : %s\n", sa[i].srcIP);
                    printf("Destination IP : %s\n", sa[i].destIP);
                    printf("Protocol       : %s\n", sa[i].protocol);
                    printf("Encryption     : %s\n", sa[i].encryption);
                }
                break;
            case 5:
                return 0;
        }
    }
}
```

Output:

```
===== IPsec Security Association Manager =====
1. Create SA
2. Lookup SA
3. Delete SA
4. Display All SAs
5. Exit

Enter your choice: 1

Enter SPI: 1001
Enter Source IP: 192.168.1.10
Enter Destination IP: IP: 192.168.1.20
Enter Protocol (AH/ESP): Enter Encryption: AES

Security Association created successfully!
```

Question 4:

```
#include <stdio.h>
#include <string.h>

long long power_mod(long long base, long long exp, long long mod)
{
    long long result = 1;

    while (exp > 0)
    {
        result = (result * base) % mod;
        exp--;
    }

    return result;
}

long long hashMessage(char message[])
{
    long long hash = 0;
    int i;

    for (i = 0; message[i] != '\0'; i++)
    {
        hash = (hash + message[i] * (i + 1)) % 100000;
    }

    return hash;
}

void encryptMessage(char message[], long long e, long long n,
                   long long encrypted[])
{

```

```

void decryptMessage(long long encrypted[], int length,
                   long long d, long long n, char decrypted[])
{
    int i;

    for (i = 0; i < length; i++)
    {
        decrypted[i] =
            (char)power_mod(encrypted[i], d, n);
    }

    decrypted[length] = '\0';
}

long long generateSignature(char message[], long long d, long long n)
{
    long long hash = hashMessage(message);

    return power_mod(hash, d, n);
}

int verifySignature(char message[], long long signature,
                   long long e, long long n)
{
    long long originalHash = hashMessage(message);
    long long decryptedSignature =
        power_mod(signature, e, n);

    return originalHash == decryptedSignature;
}

```

```

int main()
{
    char message[200];
    char decrypted[200];

    long long encrypted[200];

    long long n = 3233;

    long long receiverPublicKey = 17;
    long long receiverPrivateKey = 2753;

    long long senderPublicKey = 17;
    long long senderPrivateKey = 2753;

    long long signature;
    int length;

    printf("==== Simplified PGP System =====\n");

    printf("\nEnter Email Message: ");
    fgets(message, sizeof(message), stdin);

    message[strcspn(message, "\n")] = '\0';

    signature = generateSignature(
        message,
        senderPrivateKey,
        n
    );
}

```

```

===== Simplified PGP System =====

Enter Email Message: Hello Alice

--- Digital Signature ---
Message Hash : 6142
Digital Signature : 808

--- Encrypted Message ---
3000 1313 745 745 2185 1992 2790 745 3179 281 1313

--- Decrypted Message ---
Hello Alice

--- Signature Verification ---
Digital Signature: INVALID
Message may have been modified.

-----
Process exited after 21.81 seconds with return value 0
Press any key to continue . . . |

```

Question 5:

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

int containsWord(char text[], char word[])
{
    char lowerText[1000];
    char lowerWord[100];

    int i;

    for (i = 0; text[i] != '\0' && i < 999; i++)
        lowerText[i] = tolower((unsigned char)text[i]);

    lowerText[i] = '\0';

    for (i = 0; word[i] != '\0' && i < 99; i++)
        lowerWord[i] = tolower((unsigned char)word[i]);

    lowerWord[i] = '\0';

    return strstr(lowerText, lowerWord) != NULL;
}

int main()
{
    char from[200];
    char subject[300];
    char body[1000];
    char replyTo[200];

    int spamScore = 0;

```



```

/* Email Header Analysis */
printf("\nEnter From address: ");
fgets(from, sizeof(from), stdin);
from[strcspn(from, "\n")] = '\0';

printf("Enter Reply-To address: ");
fgets(replyTo, sizeof(replyTo), stdin);
replyTo[strcspn(replyTo, "\n")] = '\0';

printf("Enter Subject: ");
fgets(subject, sizeof(subject), stdin);
subject[strcspn(subject, "\n")] = '\0';

/* Email Content */
printf("Enter Email Body: ");
fgets(body, sizeof(body), stdin);
body[strcspn(body, "\n")] = '\0';

/* Header Analysis */

/* Check suspicious sender */
if (containsWord(from, "unknown") ||
    containsWord(from, "noreply-spam") ||
    containsWord(from, "freeoffer"))
{
    spamScore += 2;
}

/* Check Reply-To mismatch */
if (strcmp(from, replyTo) != 0)

```

Output:

```

Enter From address: teacher@college.edu
Enter Reply-To address: teacher@college.edu
Enter Subject: Assignment Submission
Enter Email Body: Please submit your assignment before Friday.

===== ANALYSIS =====
Spam Score: 0
Classification: LEGITIMATE
Reason: No significant spam indicators detected.

-----
Process exited after 78.09 seconds with return value 0
Press any key to continue . . . |

```